



المدرسة العليا
للأساتذة - مراكش
ÉCOLE NORMALE
SUPÉRIEURE - MARRAKECH

Université Cadi Ayyad
École Normale Supérieure
Département d'Informatique

Licence d'éducation : Spécialité Enseignement Secondaire - Informatique
Semestre S6
Module : Développement Web et Mobile 2

Rapport de Projet de Fin de Module

Développement Web et Mobile 2

Réalisé par

Oussama AIT HMAD

Encadré par

Pr. Mohamed Lachgaer

Présenté devant le jury composé de

Pr. Mohamed LACHGAR

Examineur

Année universitaire : 2025–2026

Déclaration

Je soussignée, **Oussama Ait Hmad** , de l'École Normale Supérieure, Université Cadi Ayyad, Département d'Informatique, déclare que le présent rapport, y compris l'ensemble des textes, figures, tableaux, extraits de code et illustrations, constitue le fruit de mon travail personnel. Les sources externes utilisées ont fait l'objet de citations et références explicites. Je reconnais que tout manquement à cet engagement relèverait du plagiat, considéré comme une infraction académique passible de sanctions.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques, au bénéfice des futurs étudiants et chercheurs, et j'accepte qu'il soit consulté dans l'intérêt général de l'Enseignement Supérieur et de la Recherche.

Ait Hmad Oussama

12 mai 2026

Remerciements

Avant tout, je dédie ce travail à Dieu Tout-Puissant, source de ma force, de ma patience et de mes espérances. C'est à Lui que je confie mes efforts, mes doutes et mes victoires.

Je tiens à exprimer ma profonde gratitude à mon encadrant, Moncieur Mohamed LACHGAR, qui a attendu ce jour avec tant de bienveillance et de confiance. Dans chaque ligne de ce projet, dans chaque décision prise, il y a son ombre douce, son regard encourageant, son accompagnement silencieux. Ce travail est aussi un hommage à ce qu'elle a semé en moi. Merci pour votre générosité en matière de formation et d'encadrement.

Je me permets aussi de me remercier moi-même, pour avoir tenu bon face aux tempêtes, pour avoir continué à avancer quand le cœur était lourd. Ce chemin n'a pas été facile, mais je suis fière de celle que je suis devenue.

Enfin, que les membres du jury trouvent ici l'expression de ma profonde gratitude pour l'honneur qu'ils me font en acceptant d'évaluer ce travail.

Abstract

This project, developed as part of the Bachelor of Science in Computer Science at ENS Marrakech, presents the design and implementation of a **Local Delivery Tracking System**. The application addresses the growing need for organized logistics in restricted geographical areas, such as university campuses or local neighborhoods, by synchronizing three complementary platforms.

The solution architecture is built around a **RESTful API** developed in **PHP**, managing a optimized database limited to four essential tables : Users, Addresses, Deliveries, and Status Logs. This backend ensures secure data flow and real-time synchronization between the various actors. The **Android mobile application** provides a dedicated interface for clients to request deliveries and for couriers to manage their routes and update statuses. Simultaneously, the **React-based web dashboard** offers administrators a powerful supervision tool, featuring real-time tracking, data management (CRUD), and key performance indicators.

Designed using **UML methodologies**, the system prioritizes security through **JWT authentication** and provides a professional user experience with responsive interfaces. By automating the coordination between delivery personnel and customers, this project contributes to the field of educational technology and smart logistics, offering a realistic, scalable, and efficient solution for local service management.

Keywords : REST API, PHP, Android, React.js, Local Delivery, Real-time Tracking, MySQL, JWT Security, Logistics Management, UML Modeling.

Résumé

Le projet présenté dans ce rapport, réalisé dans le cadre de la licence en informatique à l'ENS Marrakech, concerne la conception et la mise en œuvre d'un **système de suivi de livraison locale**. Cette application répond au besoin croissant d'organiser la logistique au sein de zones géographiques restreintes, telles qu'un campus universitaire ou un quartier, en synchronisant trois plateformes complémentaires.

L'architecture de la solution repose sur une **API REST** développée en **PHP**, gérant une base de données optimisée et limitée à quatre tables essentielles : Utilisateurs, Adresses, Livraisons et Logs de statut. Ce backend assure un flux de données sécurisé et une synchronisation en temps réel entre les différents acteurs. L'**application mobile Android** offre une interface dédiée aux clients pour soumettre des demandes de livraison et aux livreurs pour gérer leurs tournées et mettre à jour les états d'avancement. Simultanément, le **tableau de bord web réalisé avec React** fournit aux administrateurs un outil de supervision puissant, incluant le suivi en temps réel, la gestion des données (CRUD) et des indicateurs de performance.

Conçu selon les méthodologies **UML**, le système privilégie la sécurité via l'authentification **JWT** et propose une expérience utilisateur professionnelle grâce à des interfaces responsives. En automatisant la coordination entre le personnel de livraison et les clients, ce projet contribue au domaine des technologies éducatives et de la logistique intelligente, offrant une solution réaliste, évolutive et efficace pour la gestion des services de proximité.

Mots clés : API REST, PHP, Android, React.js, Livraison Locale, Suivi en Temps Réel, MySQL, Sécurité JWT, Gestion Logistique, Modélisation UML.

Liste des abréviations

Note : Trier en ordre alphabétique

API	Application Programming Interface (Interface de Programmation d'Application) : Interface permettant l'échange de données entre le serveur PHP, l'application Android et le tableau de bord React.
REST	Representational State Transfer (Transfert d'État Représentatif) : Style d'architecture logicielle utilisé pour le développement de l'API.
JWT	JSON Web Token (Jeton Web JSON) : Méthode sécurisée pour l'authentification des utilisateurs entre le client et le serveur.
PHP	Hypertext Preprocessor (Préprocesseur d'Hypertexte) : Langage de programmation côté serveur utilisé pour développer le backend de l'application.
JS	JavaScript : Langage de programmation utilisé pour le développement de l'interface d'administration Web.
JSON	JavaScript Object Notation (Notation d'Objet JavaScript) : Format d'échange de données léger utilisé pour les communications API.
UML	Unified Modeling Language (Langage de Modélisation Unifié) : Langage de modélisation visuel utilisé pour concevoir l'architecture du système.
SQL	Structured Query Language (Langage de Requête Structuré) : Langage utilisé pour interagir avec la base de données MySQL/MariaDB.
SGBD	Système de Gestion de Base de Données : Logiciel permettant de gérer la base de données du projet.
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer) : Les quatre opérations fondamentales de gestion des données (Utilisateurs, Livraisons, etc.).
UI	User Interface (Interface Utilisateur) : Design graphique des écrans de l'application Android et de l'interface React.
UX	User Experience (Expérience Utilisateur) : Qualité de l'interaction globale de l'utilisateur avec l'application.
HTTP	HyperText Transfer Protocol (Protocole de Transfert Hypertexte) : Protocole de communication utilisé pour les requêtes réseau.
APK	Android Package Kit : Format de fichier utilisé pour la distribution et l'installation de l'application mobile sur le système Android.
SPA	Single Page Application (Application Web à Page Unique) : Modèle d'application web fluide utilisé par React pour le tableau de bord.

Table des figures

1.1	Schéma du cycle Scrum et de ses principales étapes	5
1.2	Diagramme de Gantt.	7
2.1	Architecture classique d'une application mobile connectée à une API.	9
4.1	Logo de l'UML.	16
4.2	Architecture applicative du projet.	17
4.3	Le diagramme de classes.	18
4.4	Diagramme de cas d'utilisation.	19
4.5	Diagramme de séquence "Mise à jour d'un statut de livraison".	21
5.1	Langages de programmation utilisés (Java, JS, PHP, SQL).	23
5.2	Frameworks et bibliothèques (React, Volley).	24
5.3	Système de gestion de base de données relationnelle MySQL.	25
5.4	Logo pour notre application de livraison.	26
5.5	Écran de connexion (Application Mobile Android).	27
5.6	Interface Client : Création et Suivi d'une livraison.	29
5.7	Interface Livreur : Liste des courses et mise à jour du statut.	31
5.8	Tableau de bord Admin Web (Statistiques et liste globale).	32

Liste des tableaux

1.1	Équipe Scrum du projet.	5
1.2	Les sprints du projet	6
2.1	Comparaison des systèmes de suivi de livraison.	10
3.1	Description des rôles des acteurs du système de livraison.	15
4.1	Description textuelle du Client.	20
4.2	Description textuelle du Livreur.	20
4.3	Description textuelle de l'Administrateur.	20

Table des matières

Abstract	iii
Résumé	iv
Liste des abréviations	v
Table des figures	vi
Liste des tableaux	vii
Introduction Générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Présentation du projet	2
1.2.1 Sujet du projet	2
1.2.2 Intérêt du projet	2
1.3 Problématique et état de l'existant	3
1.4 Objectifs du projet	3
1.5 Démarche de gestion de projet	4
1.5.1 Choix de la méthode de gestion du projet	4
1.5.2 Planification du projet	5
1.6 Conclusion	7
2 État de l'art	8
2.1 Introduction	8
2.2 Concepts fondamentaux	8
2.2.1 Architectures Orientées Services et API REST	8
2.2.2 Développement Mobile et Expérience Utilisateur (UX)	8
2.2.3 Tableaux de bord Web et Supervision	9
2.2.4 Logistique du dernier kilomètre et suivi d'état	9
2.3 Travaux connexes et solutions existantes	9
2.4 Analyse Comparative des Solutions Existantes	10
2.5 Conclusion	10
3 Étude préliminaire et fonctionnelle	12
3.1 Introduction	12
3.2 Étude des besoins	12
3.2.1 Besoins fonctionnels	13
3.2.2 Besoins non fonctionnels	14
3.2.3 Identification des acteurs	14
3.3 Conclusion	15

4	Analyse et conception	16
4.1	Introduction	16
4.2	Le formalisme UML	16
4.3	Architecture applicative	17
4.4	Diagrammes de conception	18
4.4.1	Diagramme de classes	18
4.4.2	Diagrammes de cas d'utilisation	18
4.4.3	Diagrammes de séquence	20
4.5	Conclusion	21
5	Technologies et réalisation	22
5.1	Environnement de développement technique	22
5.1.1	Langages de programmation	22
5.1.2	Frameworks et Bibliothèques	23
5.1.3	Base de données	24
5.1.4	Critères de sélection	25
5.2	Environnement de développement logiciel	25
5.3	Identité visuelle du projet	25
5.4	Implémentation et interfaces	26
5.5	Stratégie de tests	32
5.5.1	Tests de l'API (Postman)	32
5.5.2	Tests d'intégration Mobile	32
5.5.3	Tests de bout-en-bout (Scénario global)	33
5.6	Conclusion	33
	Conclusion générale	34
	Bibliographie	36
	Appendices	38

Introduction Générale

Avec l'urbanisation croissante et l'évolution des habitudes de consommation, le secteur de la logistique de proximité connaît aujourd'hui une transformation majeure. Parmi les innovations technologiques, les systèmes de suivi en temps réel se démarquent par leur capacité à optimiser les flux de marchandises et à instaurer un climat de confiance entre les différents acteurs. Ces solutions, autrefois réservées aux grandes entreprises internationales, deviennent désormais essentielles à l'échelle locale, notamment au sein des campus universitaires ou des quartiers urbains, pour faciliter les services de livraison au dernier kilomètre.

Le projet que je présente s'inscrit dans cette dynamique de modernisation des services de proximité. Il vise à concevoir un système de suivi de livraison locale intelligent qui ne se contente pas de mettre en relation des clients et des livreurs, mais qui assure une gestion rigoureuse et transparente de chaque étape du processus. À travers une architecture multi-plateforme, ce système permet de coordonner les commandes, de suivre les déplacements et de notifier les utilisateurs sur l'avancement de leurs livraisons.

La problématique à laquelle je souhaite apporter une réponse est cruciale : comment assurer une gestion efficace, sécurisée et transparente des livraisons à une échelle locale ? Le manque de coordination et l'absence de visibilité sur le statut des colis peuvent entraîner des retards, des pertes et une insatisfaction générale. C'est dans cette optique que notre application a été pensée, comme un outil de médiation robuste capable de centraliser les informations pour offrir un service fiable en temps réel.

L'objectif principal de ce travail est donc double : d'une part, développer une infrastructure backend solide pour sécuriser les données et historiser les changements de statuts ; d'autre part, fournir des interfaces adaptées à chaque rôle — une application mobile ergonomique pour les clients et livreurs, et un tableau de bord web pour l'administration. Ce projet vise également à simplifier la prise en charge des colis, à optimiser les trajets des livreurs et à offrir aux administrateurs une vision globale de l'activité.

Pour atteindre ces buts, une méthodologie structurée a été adoptée. Elle débute par une analyse des besoins logistiques locaux et une phase de modélisation rigoureuse à l'aide des diagrammes UML pour respecter la contrainte d'une base de données optimisée. Enfin, le développement s'est appuyé sur le langage PHP pour l'API backend, sur React pour l'interface de supervision web, et sur Android pour la partie mobile.

En termes de résultats attendus, j'espère démontrer qu'une solution logicielle intégrée peut transformer la gestion des livraisons locales en un processus fluide et professionnel. L'objectif est de poser les bases d'un système évolutif, capable de s'adapter à des environnements variés tout en garantissant une qualité de service optimale.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce premier chapitre décrit le contexte général du projet, en élaborant une étude approfondie des applications qui existent en précisant l'accent sur leurs insuffisances, afin de mettre en clair les principaux besoins et objectifs à atteindre ainsi que la méthodologie à suivre avec un détail sur la répartition des tâches à respecter pour un travail à organisation optimale.

1.2 Présentation du projet

1.2.1 Sujet du projet

Développement d'une plateforme intégrée de gestion et de suivi de livraison locale destinée à l'optimisation des services logistiques de proximité. Ce système innovant combine trois environnements technologiques complémentaires : une application mobile **Android** pour l'interaction sur le terrain, une interface web en **React** pour la supervision administrative, et une architecture backend robuste en **PHP** gérant une base de données optimisée.

Le système utilise des mécanismes de **synchronisation en temps réel** et des algorithmes de gestion d'états pour suivre le parcours des colis, de la création de la commande à la livraison finale. En intégrant des fonctionnalités de **géolocalisation** et de gestion des adresses, la plateforme permet de connecter efficacement les clients et les livreurs tout en assurant une transparence totale. Le backend sécurisé via **JWT (JSON Web Token)** garantit l'intégrité des données et une gestion stricte des rôles utilisateurs (Client, Livreur, Administrateur).

La plateforme propose également un tableau de bord intelligent qui exploite l'historisation des données (**logs de statut**) pour générer des indicateurs de performance, tels que le volume de livraisons et les délais de traitement. En automatisant la coordination logistique et en fournissant un suivi proactif, ce projet vise à transformer l'expérience de livraison au dernier kilomètre, tout en offrant une solution scalable et sécurisée adaptée aux besoins réels des environnements urbains ou universitaires.

1.2.2 Intérêt du projet

Ce projet présente plusieurs avantages significatifs pour la gestion logistique de proximité :

- **Optimisation de la logistique locale** : En centralisant les demandes de livraison et en permettant un suivi en temps réel, la plateforme fluidifie les échanges entre clients et livreurs. Elle élimine les incertitudes liées à la localisation des colis et garantit une distribution plus rapide et mieux organisée au sein du campus ou du quartier.
- **Transparence et traçabilité** : Grâce à l'historisation des changements de statuts (Logs), le système offre une visibilité complète sur le parcours de chaque livraison. Cette capacité permet

d'identifier immédiatement l'étape où se trouve un colis, renforçant ainsi la confiance des utilisateurs et la responsabilité des livreurs.

- **Accessibilité multi-plateforme** : L'écosystème combinant une application mobile Android et un tableau de bord Web React assure une accessibilité optimale. Les agents sur le terrain disposent d'un outil mobile ergonomique pour gérer leurs tâches, tandis que les administrateurs bénéficient d'une interface web puissante pour superviser l'activité globale depuis n'importe quel poste.
- **Gain de temps et d'efficacité administrative** : L'automatisation des processus (création de commandes, mise à jour des statuts, calcul des statistiques) réduit considérablement la charge de travail manuel. Les administrateurs peuvent se concentrer sur l'optimisation du service plutôt que sur la gestion fastidieuse de documents papier ou d'appels téléphoniques incessants.
- **Aide à la décision par la donnée** : Les données collectées par le système permettent de générer des indicateurs de performance clés (KPI), tels que le nombre de livraisons quotidiennes ou les délais moyens de traitement. Ces informations constituent une base précieuse pour adapter les ressources (nombre de livreurs, horaires) et améliorer continuellement la qualité du service.
- **Sécurité et intégrité des données** : Le projet accorde une importance capitale à la protection des informations. L'utilisation de jetons de sécurité (**JWT**) et le hachage des mots de passe garantissent que seules les personnes autorisées accèdent aux données sensibles, créant ainsi un environnement numérique sûr pour tous les acteurs.

En réunissant ces avantages, le projet transforme la gestion des livraisons locales en une solution moderne, centrée sur les besoins des utilisateurs, qui renforce l'efficacité opérationnelle et la satisfaction au sein de la communauté locale.

1.3 Problématique et état de l'existant

Actuellement, la gestion des livraisons au sein d'environnements restreints (comme un campus ou un quartier) repose souvent sur des processus informels ou manuels. L'absence d'un système centralisé entraîne plusieurs difficultés : un manque de visibilité sur l'état des colis, des erreurs dans la transmission des adresses et une coordination difficile entre les livreurs et les clients. L'existant manque de traçabilité, ce qui peut générer des pertes de temps et une insatisfaction des utilisateurs.

1.4 Objectifs du projet

Les objectifs offrent un cadre structurant pour l'ensemble du travail. Ils peuvent être :

- **Gestion des livraisons en temps réel** :
 - Permettre aux clients de créer des demandes de livraison avec des adresses précises.
 - Permettre aux livreurs de prendre en charge les colis et de mettre à jour leur progression.
 - Assurer le suivi instantané du colis depuis la collecte jusqu'à la destination finale.
- **Suivi et Historisation (Logs)** :
 - Enregistrement automatique de chaque changement de statut (Créé, En route, Livré).
 - Consultation de l'historique des étapes pour garantir la traçabilité et résoudre les litiges.
- **Administration et Supervision** :
 - Interface web centralisée pour la gestion (CRUD) des utilisateurs, des adresses et des commandes.
 - Tableau de bord affichant des indicateurs clés (nombre de livraisons, livreurs actifs).

1.4.2 Objectifs Techniques

- **Architecture Multi-plateforme :**
 - Développement d'une API REST robuste en **PHP** pour centraliser les données.
 - Application mobile **Android** (Java/Kotlin) pour les utilisateurs nomades.
 - Interface d'administration en **React.js** pour une gestion fluide et réactive.
- **Sécurité et Intégrité :**
 - Authentification sécurisée basée sur les jetons **JWT**.
 - Protection des données sensibles et validation stricte des formulaires côté serveur.
- **Performance et Optimisation :**
 - Temps de réponse de l'API inférieur à 1 seconde pour garantir une expérience mobile fluide.
 - Base de données SQL optimisée respectant la contrainte pédagogique de **4 tables**.

1.5 Démarche de gestion de projet

1.5.1 Choix de la méthode de gestion du projet

Afin de bien cerner les tâches à accomplir, nous avons adopté la méthode Scrum, une approche collaborative reconnue pour sa gestion efficace des projets informatiques. Basée sur des échanges constructifs entre toutes les parties prenantes lors de réunions régulières, cette méthode favorise une communication transparente et une adaptation continue aux besoins du client.

La méthode Scrum (1) est un cadre de gestion de projet qui entre dans la catégorie des méthodes agiles. Les méthodes agiles privilégient la flexibilité dans le processus de développement, l'implication du client et la prise en compte de ses retours tout au long du projet.

Contrairement aux méthodes classiques de gestion de projet, Scrum définit trois rôles principaux :

- **Le Scrum Master** a pour rôle :
 - De vérifier l'application des principes Scrum
 - D'assurer une communication fluide au sein de l'équipe
 - D'identifier des pistes d'amélioration pour la productivité
- **L'équipe Scrum** est composée :
 - D'une dizaine de personnes (variable selon la taille du projet)
 - De membres auto-organisés sans hiérarchie
 - De profils complémentaires travaillant en collaboration
- **Le Product Owner** a pour missions :
 - D'être l'expert métier du client
 - De définir les spécifications fonctionnelles
 - D'arbitrer les priorités de réalisation

La figure 1.1 présente le schéma du cycle Scrum et de ses principales étapes.

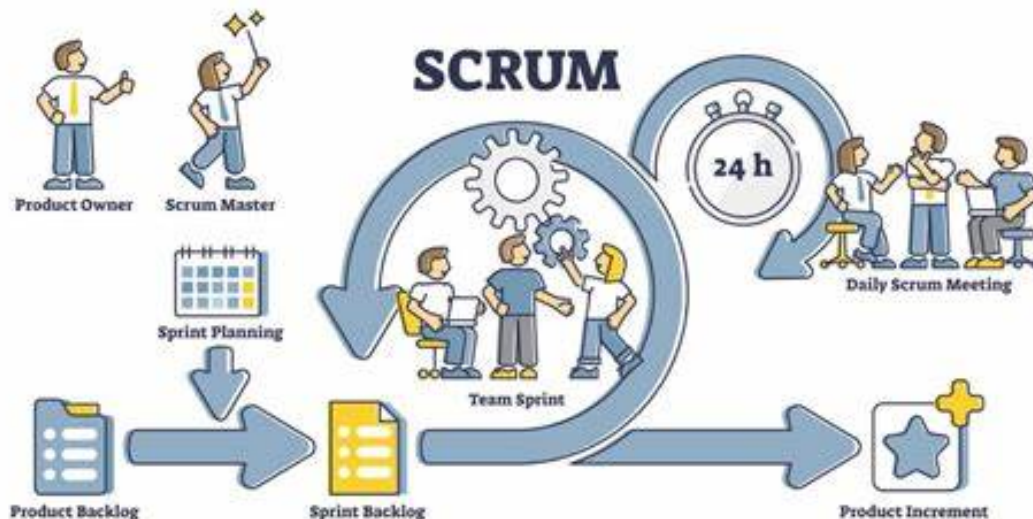


Figure 1.1 : Schéma du cycle Scrum et de ses principales étapes .

1.5.2 Planification du projet

Équipe SCRUM

Une équipe Scrum est une équipe interfonctionnelle, productive et auto-organisée qui vise à fournir des produits de haute qualité. Elle se compose d'un Scrum Master, d'un Product Owner et d'une équipe de développement.

L'équipe Scrum de ce projet est présentée dans le tableau 1.1.

Table 1.1 : Équipe Scrum du projet.

Rôle	Personne en charge
Scrum Product Owner	Pr. Mohamed LACHGAR
Scrum Master	Oussama AIT HMAD
Scrum Development Team	Oussama AIT HMAD

Sprints

Après le choix de l'équipe scrum et la répartition des rôles, il est maintenant le temps de préparer les sprints et de définir leurs objectifs (Increment), et c'est exactement le travail effectué que vous pouvez voir sur le tableau 1.2.

Table 1.2 : Les sprints du projet .

Sprint	Objectifs et tâches
Sprint 1	Étude et conception : ▪ Analyse des besoins du système de livraison. ▪ Création des diagrammes UML. ▪ Conception des maquettes (Mobile et Web).
Sprint 2	Base de données et Backend : ▪ Création de la base de données (4 tables max). ▪ Mise en place de l'architecture serveur. ▪ Développement des API REST de base.
Sprint 3	Développement frontend : ▪ Implémentation des interfaces Client (Mobile). ▪ Implémentation des interfaces Livreur (Mobile). ▪ Développement des interfaces Admin (Web React).
Sprint 4	Connexion et Authentification : ▪ Développement de l'inscription/connexion. ▪ Sécurisation des API par JWT. ▪ Gestion des profils utilisateurs.
Sprint 5	Fonctionnalités Client (Mobile) : ▪ Formulaire de demande de livraison. ▪ Suivi des statuts (créée, en route, livrée...). ▪ Consultation de l'historique des livraisons.
Sprint 6	Fonctionnalités Livreur (Mobile) : ▪ Affichage des livraisons affectées. ▪ Mise à jour de l'état des livraisons. ▪ Affichage des destinations.
Sprint 7	Fonctionnalités Administrateur (Web) : ▪ Gestion globale et affectation des livraisons. ▪ Gestion des adresses fréquentes. ▪ Tableau de bord d'analyse et statistiques.
Sprint 8	Tests et optimisation : ▪ Tests du cycle de vie complet d'une livraison. ▪ Correction des bugs et optimisation. ▪ Vérification des règles métier.
Sprint 9	Rédaction du rapport : ▪ Rédaction du dossier de conception technique. ▪ Documentation des endpoints de l'API. ▪ Préparation des supports de soutien.
Sprint 10	Validation finale : ▪ Revue complète du projet. ▪ Déploiement de l'API et du Web. ▪ Génération de l'application (APK Android).

Diagramme de GANTT

La phase de planification est essentielle dans l'élaboration d'un projet. Elle implique la définition et l'organisation minutieuse des diverses tâches constituant le projet, en évaluant le temps et les ressources requis pour leur accomplissement.

J'ai employé Gantt, l'un des outils de planification de projet à ma disposition. Ce modèle est fréquemment utilisé dans la gestion de projet et est parmi les outils les plus performants pour illustrer

visuellement la progression des diverses tâches qui forment un projet.

La colonne de gauche du graphique détaille toutes les tâches à réaliser, tandis que la ligne supérieure indique les unités de temps les plus appropriées pour le projet (jours, semaines, mois, etc.). Chaque tâche est représentée par une barre horizontale, dont la position et la longueur représentent la date de début, la durée et la date de fin.

La figure 1.2 représente le planning du projet depuis l'étude d'existence jusqu'au dernier sprint en utilisant Gantt.

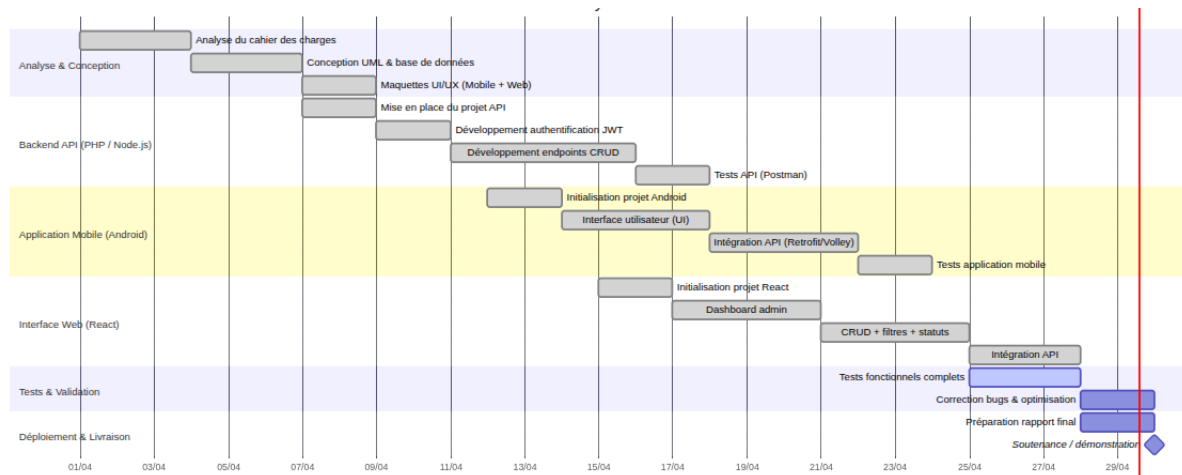


Figure 1.2 : Diagramme de Gantt.

1.6 Conclusion

Ce chapitre établit les fondements du projet en présentant son contexte général. Il débute par une description claire de la problématique à résoudre et des objectifs visés. Par la suite, il explicite l'approche méthodologique adoptée pour réaliser ces objectifs, tout en mettant l'accent sur la méthodologie de développement choisie. Le chapitre suivant se concentrera sur l'analyse initiale et fonctionnelle du projet.

Chapitre 2

État de l'art

2.1 Introduction

Ce chapitre expose les notions essentielles et les technologies principales mises en œuvre dans le développement d'un système multi-plateforme de suivi de livraison locale. Il présente les concepts des architectures orientées services, des API REST, du développement mobile natif et des interfaces web d'administration. Par la suite, une analyse des solutions existantes dans le secteur de la logistique et de la livraison du dernier kilomètre est réalisée, ce qui permet de positionner notre projet par rapport au paysage actuel des applications de livraison.

2.2 Concepts fondamentaux

2.2.1 Architectures Orientées Services et API REST

L'architecture orientée services (SOA) est un modèle de conception logicielle où les fonctionnalités sont divisées en services indépendants communiquant via un réseau. Dans notre contexte, l'utilisation d'une API (Application Programming Interface) RESTful joue un rôle central. Elle permet à l'application mobile (Android) et au tableau de bord (Web) de communiquer de manière fluide et sécurisée avec le serveur central (Backend en PHP/Node.js) via des requêtes HTTP et le format de données JSON.

2.2.2 Développement Mobile et Expérience Utilisateur (UX)

Le développement mobile, particulièrement sur Android, est essentiel pour connecter les acteurs sur le terrain (clients et livreurs). Les applications mobiles modernes nécessitent une interface utilisateur (UI) intuitive et réactive. Pour les livreurs, l'application doit permettre une mise à jour rapide des statuts de livraison, tandis que pour les clients, elle doit offrir une visibilité claire et en temps réel de l'état de leur commande. L'intégration de bibliothèques de requêtes asynchrones (comme Retrofit ou Volley) est fondamentale pour garantir une expérience sans latence.

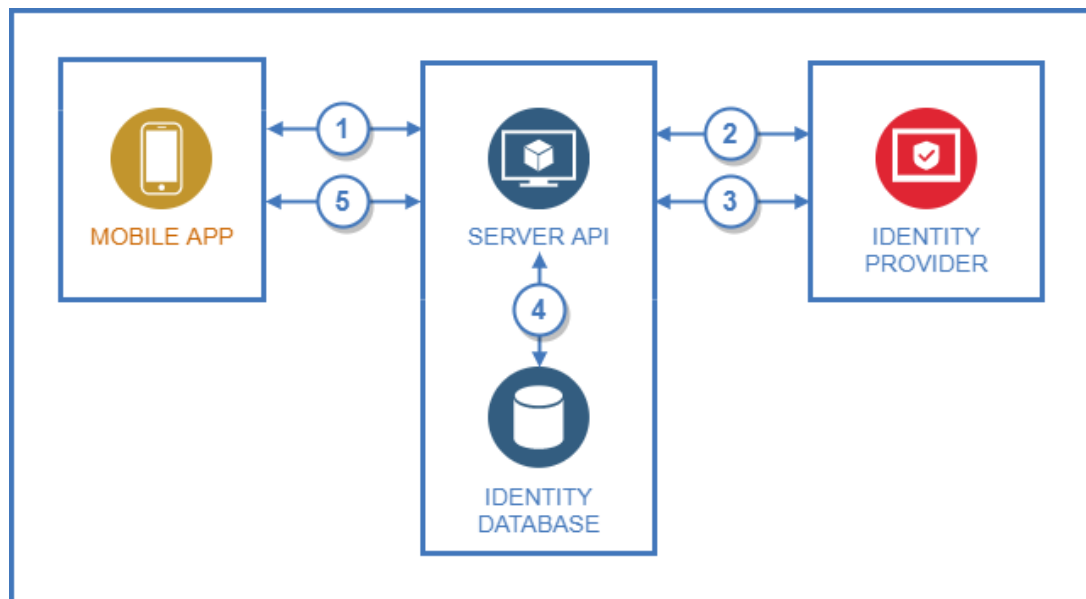


Figure 2.1 : Architecture classique d'une application mobile connectée à une API.

2.2.3 Tableaux de bord Web et Supervision

Les tableaux de bord (Dashboards) développés avec des technologies modernes comme React permettent aux administrateurs de superviser les opérations en temps réel. Ils agrègent les données issues de la base de données pour afficher des indicateurs de performance clés (KPI) tels que le nombre de livraisons du jour, les délais moyens ou les taux d'annulation. Ces interfaces permettent également l'affectation manuelle et la gestion des entités (CRUD).

2.2.4 Logistique du dernier kilomètre et suivi d'état

La "logistique du dernier kilomètre" désigne l'étape finale du processus de livraison, souvent la plus coûteuse et complexe. Un système de suivi efficace repose sur une gestion rigoureuse des "états" (statuts) d'une livraison (ex : Créée, Prise en charge, En route, Livrée, Annulée). Chaque transition d'état doit être historisée pour garantir la traçabilité et résoudre les éventuels litiges.

2.3 Travaux connexes et solutions existantes

Les géants de la livraison (Uber Eats, Glovo, Deliveroo)

Ces plateformes ont démocratisé le suivi de livraison en temps réel. Elles utilisent des algorithmes complexes de dispatching automatique, de la géolocalisation continue (GPS) et des interfaces hautement optimisées. Cependant, ces solutions sont propriétaires, fermées, et prélèvent des commissions importantes, ce qui les rend inadaptées pour une petite entreprise locale souhaitant gérer sa propre flotte de coursiers.

Solutions SaaS de gestion de flotte (Tookan, Onfleet)

Des plateformes comme Onfleet offrent des solutions "Software as a Service" pour la gestion des livraisons. Elles proposent des tableaux de bord avancés et des applications pour les chauffeurs. Bien que très complètes, elles sont souvent surdimensionnées et coûteuses pour de petites structures locales qui n'ont besoin que des fonctionnalités de base (affectation et suivi de statut).

Limites et défis des solutions existantes

Malgré leurs avantages, plusieurs défis persistent pour les petites structures :

- **Coût d'intégration élevé** : Les solutions SaaS ou les géants imposent des abonnements ou des commissions.
- **Complexité inutile** : De nombreuses fonctionnalités (optimisation algorithmique d'itinéraires) sont superflues pour un besoin local simple.
- **Personnalisation limitée** : Il est difficile d'adapter ces systèmes fermés à des règles métier spécifiques (comme des points de livraison fréquents type campus).
- **Dépendance à la géolocalisation continue** : Qui consomme énormément de batterie sur les téléphones des livreurs, là où un simple changement de statut manuel pourrait suffire.

2.4 Analyse Comparative des Solutions Existantes

Table 2.1 : Comparaison des systèmes de suivi de livraison.

Solution	Public cible	Modèle économique	Complexité technique	Pertinence pour petite structure locale
Uber / Glovo	Grand public / Restauration	Commissions sur ventes	Très élevée (IA, dispatching auto, GPS live)	Faible (Coût élevé, système fermé)
Onfleet	Moyennes et grandes entreprises	Abonnement SaaS (coûteux)	Élevée (Intégration API complexe)	Moyenne (Trop de fonctionnalités)
Suivi classique (Collissimo)	E-commerce national	Paieement à l'envoi	Basse (Scan de codes-barres aux dépôts)	Inadapté (Pas de livraison instantanée)
Notre Projet (Solution sur-mesure)	Petits commerces, campus, entreprises locales	Open-source / Interne	Adaptée (Architecture simple, 4 tables, statuts manuels)	Très forte (Répond exactement au besoin sans surcoût)

L'analyse comparative des plateformes existantes montre que les solutions actuelles sont soit trop onéreuses et complexes (Uber, Onfleet), soit inadaptées à la livraison locale instantanée (suivi postal classique).

Notre projet, axé sur la **conception d'un système de suivi de livraison locale léger**, vise à combler ce vide. En imposant une architecture claire et minimaliste (limitée à 4 tables de base de données), l'objectif est de fournir les fonctionnalités essentielles (création de livraison, affectation, mise à jour des statuts, tableau de bord) sans la lourdeur des systèmes commerciaux.

2.5 Conclusion

Ce chapitre consacré à l'état de l'art nous a permis d'explorer les architectures logicielles requises pour notre système (Mobile, React, API) et d'analyser les solutions logistiques existantes sur le

marché. En identifiant la complexité et le coût des systèmes actuels, nous avons justifié la pertinence de développer une application légère, centralisée sur les statuts de livraison, et parfaitement adaptée à un usage local. Le prochain chapitre sera dédié à l'analyse et à la conception (UML) de notre plateforme, en détaillant les interactions entre les clients, les livreurs et l'administrateur.

Chapitre 3

Étude préliminaire et fonctionnelle

3.1 Introduction

L'étude fonctionnelle constitue une étape importante dans le développement de notre système de suivi de livraison locale. Elle implique une analyse méticuleuse des besoins des utilisateurs afin de cerner les fonctionnalités attendues du système, de respecter les contraintes techniques du module (API, Mobile Android, Web React), et de définir des spécifications fonctionnelles précises.

3.2 Étude des besoins

L'étude des besoins vise à évaluer la faisabilité du projet et à proposer des solutions adéquates aux besoins logistiques de livraison locale. En s'appuyant sur les analyses effectuées, j'ai identifié trois acteurs physiques clés dont la participation est essentielle à la réussite de ce projet, ainsi que le système central. Les acteurs identifiés sont :

Administrateur (Interface Web React)

Ce module permet à l'administrateur de **gérer l'ensemble de la plateforme** et de superviser les opérations de livraison.

Une fois authentifié sur le tableau de bord Web, l'administrateur a accès au menu suivant :

- **Tableau de bord (Dashboard)** : l'administrateur peut consulter des statistiques telles que :
 - le nombre de livraisons du jour (en cours, terminées, annulées),
 - le délai moyen de livraison,
 - l'activité des livreurs.
- **Gestion des livraisons** : visualiser toutes les demandes et affecter manuellement une livraison à un livreur si nécessaire.
- **Gestion des adresses** : gérer les points de livraison fréquents (campus, quartiers).
- **Déconnexion** : pour fermer sa session de manière sécurisée.

Client (Application Mobile Android)

Ce module permet au client de créer des demandes de livraison et de suivre leur acheminement en temps réel.

Une fois authentifié, le client dispose des menus suivants :

- **Profil** : Le client peut consulter et modifier ses informations personnelles.
- **Nouvelle livraison** : le client peut créer une demande en spécifiant l'adresse de départ et de destination.
- **Suivi en temps réel** : le client peut visualiser le statut actuel de sa livraison (créée, prise en charge, en route, livrée).
- **Historique** : le client a accès à l'historique complet de ses anciennes commandes.
- **Déconnexion** : pour fermer sa session.

Livreur (Application Mobile Android)

Ce module permet au livreur d'organiser sa tournée et de mettre à jour l'état des colis qu'il transporte.

Une fois authentifié, le livreur dispose des menus suivants :

- **Livraisons affectées** : consulter la liste des livraisons qui lui sont assignées.
- **Détails et itinéraire** : voir les détails du client, l'adresse source et la destination.
- **Mise à jour des statuts** : modifier l'état de la livraison (ex : passer de "Prise en charge" à "En route", puis à "Livrée").
- **Déconnexion** : pour fermer sa session.

Système Central (API Backend)

Le système central (développé en PHP ou Node.js) est responsable du traitement des données, de la sécurité et de la persistance dans la base de données (limitée à 4 tables).

Il effectue automatiquement les actions suivantes :

- **Sécurisation** : générer et valider les tokens JWT pour sécuriser les appels API.
- **Historisation des statuts** : enregistrer automatiquement chaque changement de statut d'une livraison dans une table de logs dédiée (*delivery_status_logs*).
- **Validation métier** : vérifier qu'une livraison possède bien une adresse de départ et d'arrivée avant validation.

3.2.1 Besoins fonctionnels

Cette phase permet de déterminer les objectifs et les fonctionnalités que le système doit fournir, en respectant les contraintes du cahier des charges.

Les besoins fonctionnels se déclinent en plusieurs fonctionnalités essentielles :

- **Authentification** : Permettre aux 3 types d'utilisateurs de se connecter de manière sécurisée (JWT).
- **Création de livraison** : Le client doit pouvoir formuler une demande de course avec un point A et un point B.
- **Mise à jour d'état** : Le livreur doit pouvoir signaler l'avancement de la livraison via des statuts prédéfinis.
- **Traçabilité** : Le système doit conserver l'historique de chaque changement de statut pour une livraison donnée.
- **Supervision globale** : L'administrateur doit pouvoir effectuer des opérations CRUD via une interface web et lire les compteurs statistiques.

Illustration par des scénarios d'utilisation concrets :

- **Création et suivi** : Un client ouvre son application mobile, renseigne une adresse de récupération sur le campus et une adresse de livraison. Le système enregistre la livraison avec le statut "Créée". Le client voit sa demande en attente.
- **Prise en charge** : Un livreur se connecte à son application mobile, voit la demande, et appuie sur "Prendre en charge". L'API enregistre ce nouveau statut. Le client voit instantanément sur son écran que son colis est en cours de préparation.
- **Supervision administrative** : L'administrateur, depuis son navigateur Web (React), remarque qu'une livraison est en statut "Créée" depuis trop longtemps. Il utilise l'interface pour affecter manuellement cette livraison à un livreur disponible.

3.2.2 Besoins non fonctionnels

La capture des besoins non fonctionnels permet de définir les exigences de qualité du système, qui influencent directement la satisfaction des utilisateurs et le respect des contraintes techniques du module d'enseignement. Les besoins non fonctionnels du système englobent plusieurs aspects essentiels :

- **Ergonomie Mobile** : L'interface des applications Android doit être intuitive, avec des boutons larges et accessibles, particulièrement pour le livreur qui l'utilise en mobilité.
- **Sécurité** : Les mots de passe doivent être hachés en base de données, et toutes les requêtes de l'application mobile et du Web doivent passer par l'authentification JWT.
- **Performance** : Les appels API (via Retrofit ou Volley sur Android) doivent inclure une gestion d'état (chargement, succès, erreur) pour ne pas bloquer l'utilisateur.
- **Simplicité structurelle** : Le système doit strictement respecter la limite pédagogique de 4 tables dans la base de données relationnelle.
- **Responsive Design** : L'interface web administrateur doit s'adapter correctement aux différentes tailles d'écrans d'ordinateurs et de tablettes.

3.2.3 Identification des acteurs

L'identification et la modélisation des acteurs constituent une étape essentielle. Dans le cadre de ce système, divers acteurs interviennent et jouent des rôles clés.

Le tableau [3.1](#) illustre les différents acteurs de cette application et leurs responsabilités.

Table 3.1 : Description des rôles des acteurs du système de livraison.

Acteur	Rôle
Administrateur	▪ Consulter les statistiques via le tableau de bord Web (React) ; ▪ Gérer globalement les livraisons et les affectations ; ▪ Gérer les adresses fréquentes pour faciliter la saisie.
Client	▪ Soumettre de nouvelles demandes de livraison depuis l'application Android ; ▪ Suivre l'évolution des statuts de ses commandes ; ▪ Consulter son historique personnel.
Livreur	▪ Consulter les courses qui lui sont attribuées sur son mobile ; ▪ Mettre à jour l'état d'avancement (En route, Livrée, etc.) ; ▪ Accéder aux détails des adresses de récupération et de dépôt.
API Backend	▪ Assurer la communication JSON entre le mobile et le Web ; ▪ Maintenir la cohérence de la base de données (4 tables) ; ▪ Gérer la sécurité, les accès JWT et l'historisation des logs de statut.

3.3 Conclusion

Ce chapitre a détaillé les phases importantes de l'étude préliminaire de notre projet de suivi de livraison. L'étude fonctionnelle s'est concentrée sur la collecte des besoins de nos trois types d'utilisateurs (Administrateur, Client, Livreur) en respectant les contraintes strictes du cahier des charges (plateformes spécifiques, base de données restreinte). Le chapitre suivant s'attardera sur la conception détaillée du système, notamment à travers les diagrammes UML (cas d'utilisation, classes et séquences).

Chapitre 4

Analyse et conception

4.1 Introduction

La conception est une étape primordiale pour produire une application de haute qualité. Elle a pour objectif d'élaborer des modèles détaillés de l'architecture du système à partir du modèle obtenu lors de l'étape d'analyse des besoins. Elle vise également à réduire la complexité du système. Dans ce chapitre nous allons détailler le choix conceptuel de notre plateforme de livraison locale à travers l'architecture logicielle et plusieurs types de diagrammes UML.

4.2 Le formalisme UML

L'UML (*Unified Modeling Language* ou Langage de modélisation unifiée en français) est un langage graphique de modélisation informatique. Ce langage est désormais la référence en modélisation objet, ou programmation orientée objet. Cette dernière consiste à modéliser des éléments du monde réel ou virtuel en un ensemble d'entités informatiques appelées « objets ». L'UML est constitué de diagrammes qui servent à visualiser et décrire la structure et le comportement des objets qui se trouvent dans un système. Il permet de présenter des systèmes logiciels complexes de manière plus simple et compréhensible qu'avec du code informatique.



Figure 4.1 : Logo de l'UML.

UML n'étant pas une méthode, l'utilisation des diagrammes est laissée à l'appréciation de chacun. Le diagramme de classes est généralement considéré comme l'élément central d'UML, en particulier pour concevoir notre base de données relationnelle.

Parmi les raisons pour lesquelles on a choisi ce langage de modélisation :

- L'UML assure une bonne communication entre les différents intervenants (concepteurs, développeurs) ;

- L'UML simplifie la complexité logicielle et améliore la qualité du travail ;
- Il fournit un guide strict dans la construction d'un système (particulièrement utile pour respecter notre contrainte de 4 tables) ;
- Il permet de documenter techniquement les décisions prises.

4.3 Architecture applicative

ARCHITECTURE DU PROJET - SUIVI DE LIVRAISON LOCALE (Campus Delivery)

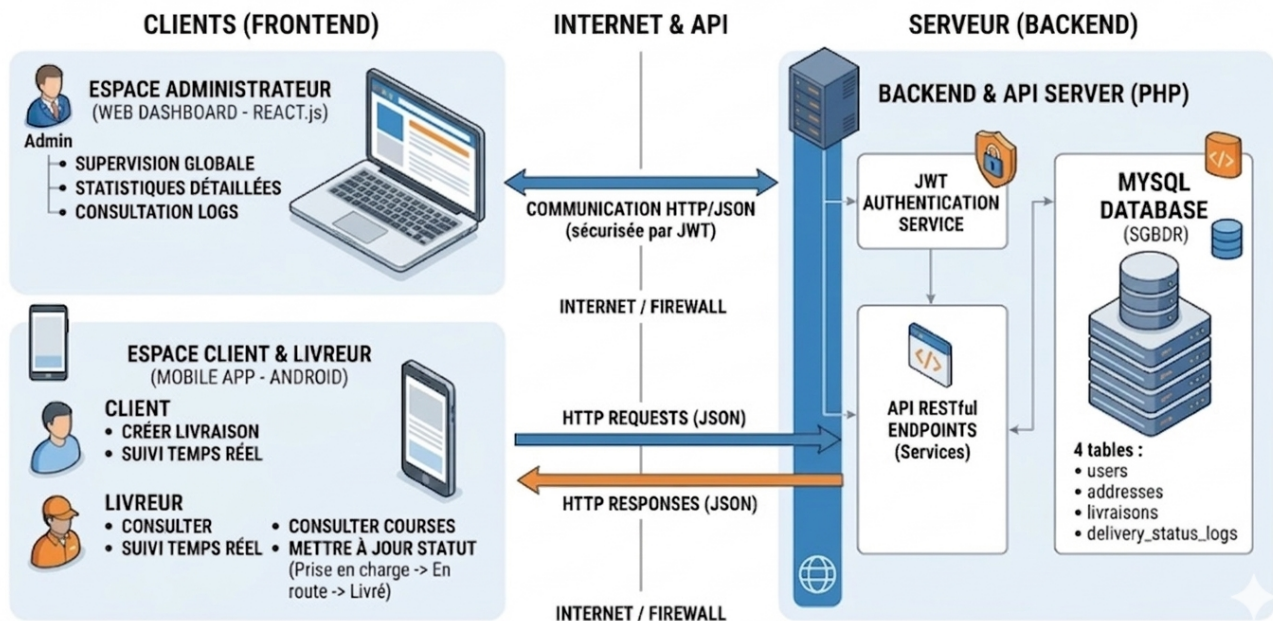


Figure 4.2 : Architecture applicative du projet.

La structure de notre application, telle qu'illustrée dans la figure 4.2, est basée sur une architecture orientée services (SOA) avec une découpe des responsabilités claire. Le système repose sur trois pôles principaux :

- **Le Frontend (Web et Mobile)** : L'interface d'administration sera développée à l'aide de la bibliothèque React (HTML5, CSS3, JavaScript), permettant à l'administrateur de gérer le système via un navigateur web. En parallèle, les clients et livreurs utiliseront une application mobile native Android.
- **Le Backend (API REST)** : Développé à l'aide de PHP, il forme le serveur qui traite la logique métier. L'API REST agit comme pont de communication, recevant les requêtes HTTP (GET, POST, PUT, DELETE) des interfaces Web et Mobiles, et renvoyant des réponses structurées au format JSON. Elle gère également la sécurité via des tokens JWT.
- **La Base de données** : Basée sur MySQL, elle gère le stockage des données critiques (utilisateurs, livraisons, statuts, adresses). Le backend interagit directement avec la base pour les opérations CRUD.

4.4 Diagrammes de conception

Cette section détaille les différents diagrammes qui ont servi à formaliser la conception du système de suivi de livraison.

4.4.1 Diagramme de classes

Le diagramme de classes est essentiel pour modéliser la structure statique du système et concevoir la base de données. Afin de respecter le cahier des charges, notre modèle se limite strictement à 4 entités (tables) majeures.

La figure 4.3 illustre les 4 classes de notre système :

1. **Users** : Gère l'authentification et les profils (Client, Livreur, Admin).
2. **Addresses** : Centralise les points de départ et d'arrivée géographiques.
3. **Deliveries** : L'entité centrale liant un client, un livreur, et deux adresses (source et destination).
4. **Delivery_status_logs** : Gère l'historisation de l'évolution de la livraison (Créée, En route, Livrée) avec horodatage.

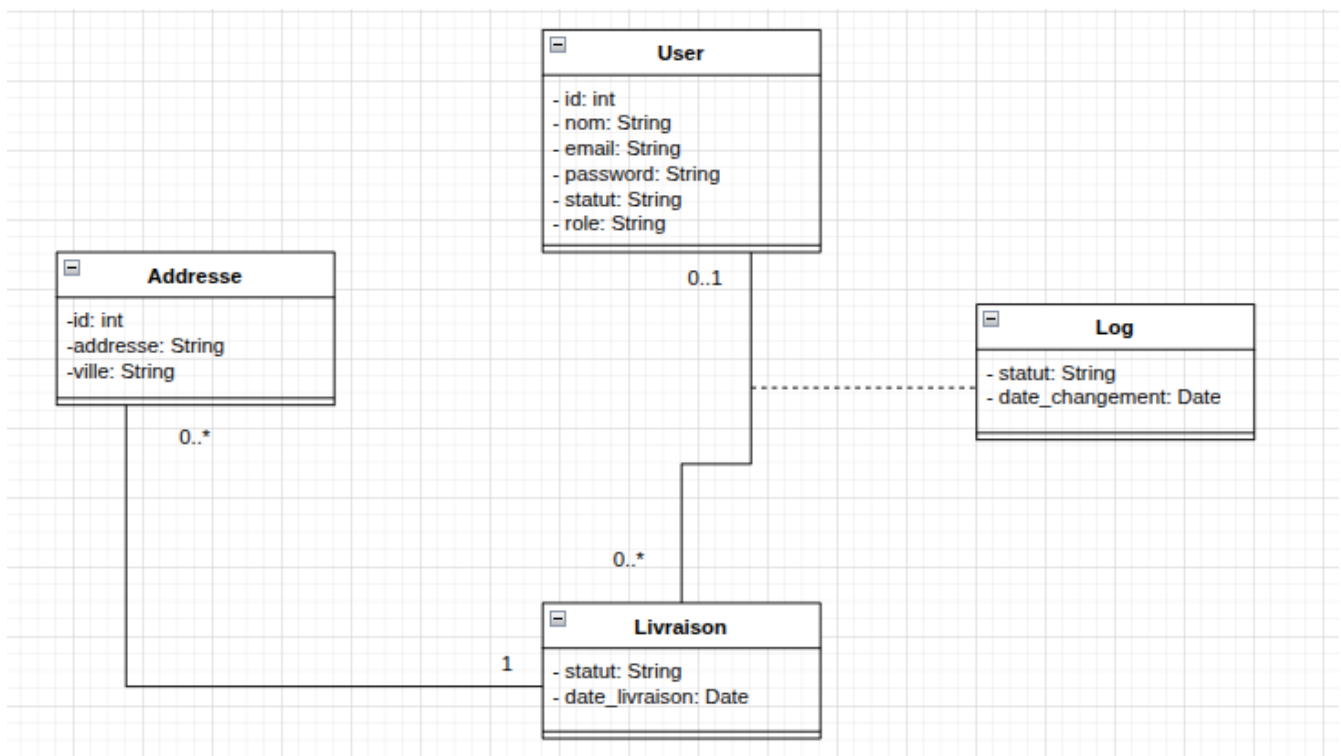


Figure 4.3 : Le diagramme de classes.

4.4.2 Diagrammes de cas d'utilisation

Les figures illustrant les cas d'utilisation représentent les interactions entre les trois différents acteurs physiques de notre plateforme de livraison.

L'**Espace Client (Mobile)** offre un environnement où le client peut créer une nouvelle demande de livraison en sélectionnant ses adresses, suivre en temps réel l'évolution du statut de son colis, et consulter l'historique de ses anciennes requêtes.

L'**Espace Livreur (Mobile)** est l'outil de travail du coursier. Il lui permet de voir la liste des livraisons qui lui sont affectées, d'accéder aux détails de l'itinéraire, et surtout, de mettre à jour le statut de la livraison (de "Prise en charge" à "Livrée") pour informer le système et le client.

L'**Espace Admin (Web)** centralise la gestion logistique. L'administrateur y consulte le tableau de bord (statistiques du jour), gère manuellement l'affectation des livraisons en attente, et ajoute ou modifie des points d'adresses fréquents.

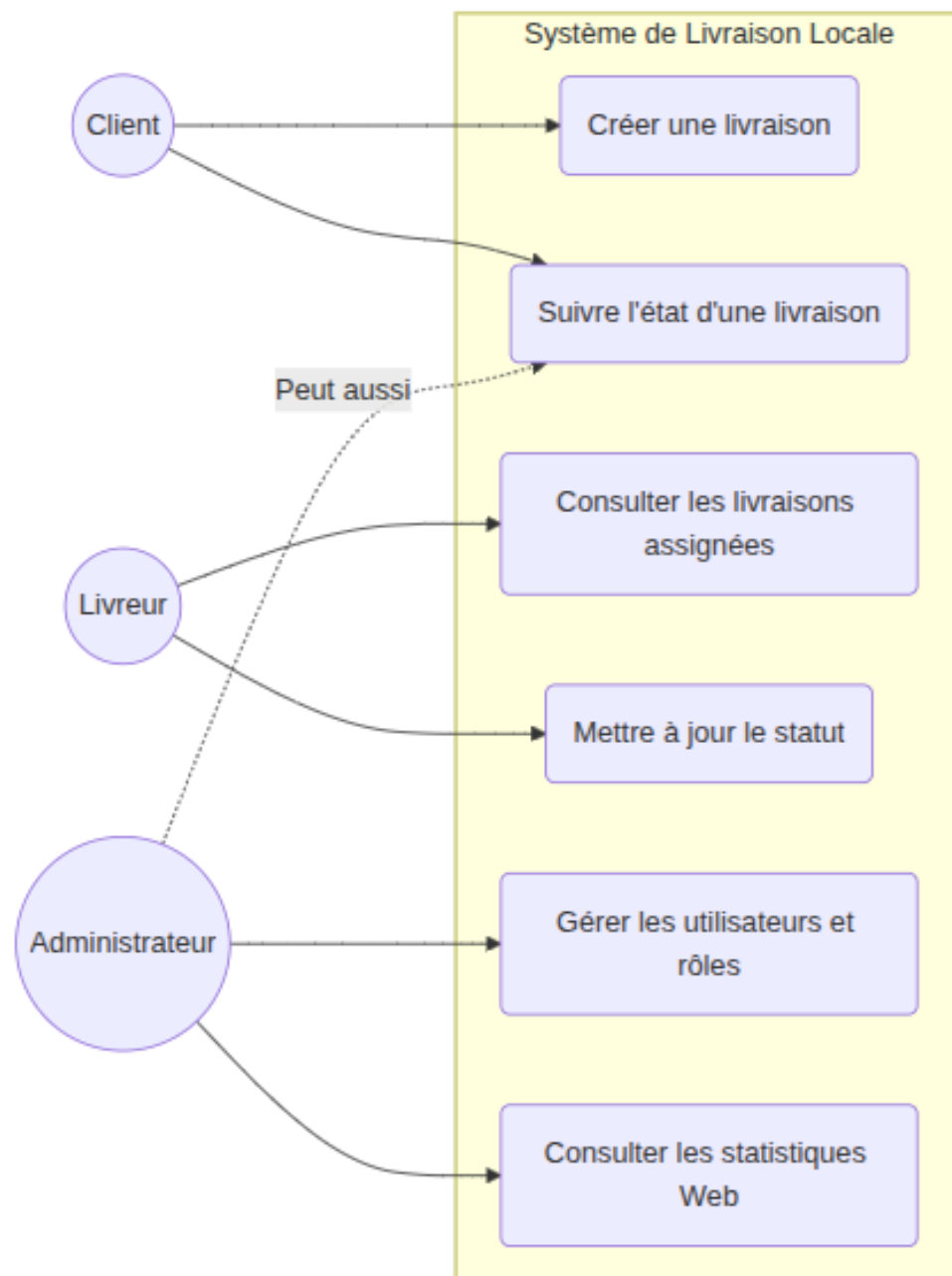


Figure 4.4 : Diagramme de cas d'utilisation.

Client

Table 4.1 : Description textuelle du Client.

Titre	Client
Résumé	Cet acteur représente les utilisateurs demandant des livraisons locales.
Préconditions	Authentification sur l'application mobile requise.
Description	Le client se connecte, remplit un formulaire de création de livraison (adresses de départ et d'arrivée), puis valide. Il peut ensuite naviguer vers la liste de ses livraisons pour observer les changements d'état (ex : "En route") effectués par le livreur, et ce jusqu'à l'état final ("Livrée" ou "Annulée").

Livreur

Table 4.2 : Description textuelle du Livreur.

Titre	Livreur
Résumé	Cet acteur représente le personnel chargé du transport des colis.
Préconditions	Authentification sur l'application mobile avec le rôle "Livreur".
Description	Le livreur consulte la liste des courses qui lui sont assignées. Il sélectionne une course, se rend au point de départ, et déclenche l'action "Prise en charge". Une fois arrivé à destination, il déclenche l'action "Livrée". Chaque action met à jour l'historique du système de manière asynchrone via l'API.

Administrateur

Table 4.3 : Description textuelle de l'Administrateur.

Titre	Administrateur
Résumé	Cet acteur supervise l'ensemble des opérations logistiques via l'interface Web.
Préconditions	Authentification sur le Dashboard Web React avec privilèges d'administration.
Description	L'administrateur accède à des indicateurs clés (compteurs de livraisons du jour, taux de succès). Si une livraison reste trop longtemps non assignée, il peut l'attribuer manuellement à un livreur. Il gère également les tables de base (CRUD sur les utilisateurs et les adresses prédéfinies).

4.4.3 Diagrammes de séquence

Le diagramme de séquence décrit comment les objets interagissent dans un certain ordre pour accomplir une tâche spécifique, notamment les appels API entre le client et le serveur.

Cycle de vie d'une livraison

La figure 4.5 présente le processus critique du changement de statut d'une livraison. Le scénario commence lorsqu'un livreur appuie sur le bouton "Colis récupéré" sur son mobile. L'application Android envoie une requête POST `/api/deliveries/status` au Backend. L'API vérifie le token

JWT, puis interroge la base de données MySQL pour insérer une nouvelle ligne dans la table *delivery_status_logs*. Une fois la transaction confirmée, le backend renvoie un code HTTP 200 (Succès) au mobile du livreur. De son côté, lorsque le Client rafraîchit son application, une requête GET est envoyée à l'API qui lui retourne le nouveau statut, mettant ainsi à jour l'interface utilisateur.

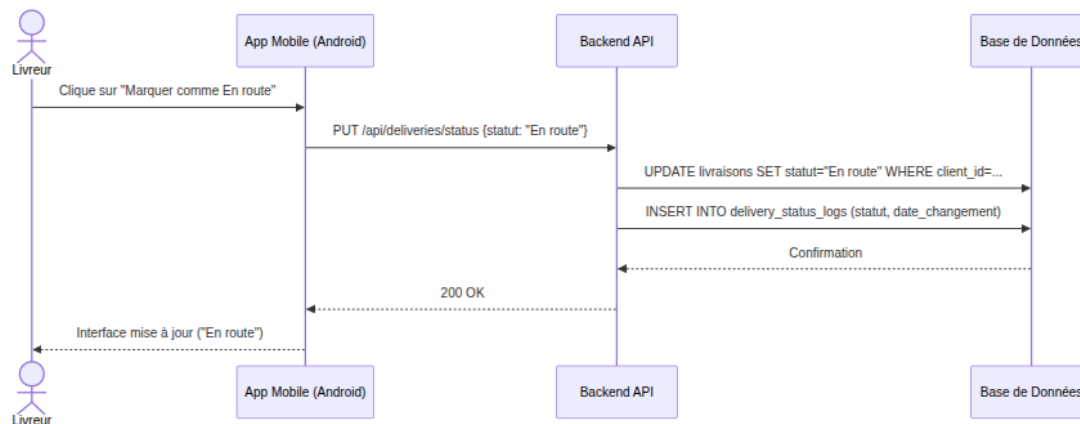


Figure 4.5 : Diagramme de séquence "Mise à jour d'un statut de livraison".

Ces diagrammes fournissent une vue d'ensemble claire et précise de la structure logicielle et des flux de données, validant la faisabilité technique sous la contrainte des 4 tables de la base de données.

4.5 Conclusion

En somme, la phase de conception a fourni une feuille de route claire pour le développement de notre système de suivi de livraison locale. Elle a permis de s'assurer que toutes les exigences du cahier des charges (découplage API/Web/Mobile, limite de 4 tables) sont bien comprises et correctement documentées via le langage UML. Le système est conçu pour être à la fois fonctionnel, léger et adapté à la logistique du dernier kilomètre.

Dans la partie suivante, nous présenterons les outils, les langages et les technologies (Android, React, Backend) utilisés dans la phase de réalisation.

Chapitre 5

Technologies et réalisation

Ce chapitre décrit l'ensemble des technologies, outils et méthodes qui ont été utilisés pour la mise en œuvre de notre plateforme de suivi de livraison locale. Il présente d'abord les choix techniques (Mobile, Web et API), puis détaille l'environnement de développement, l'implémentation du système à travers les interfaces utilisateur (Client, Livreur, Administrateur), ainsi que la stratégie de tests appliquée. Enfin, une conclusion synthétique fait le point sur les réalisations.

5.1 Environnement de développement technique

5.1.1 Langages de programmation

Java / Kotlin (Android) Pour le développement de l'application mobile native destinée aux clients et aux livreurs, nous avons utilisé Java/Kotlin avec le SDK Android. Ces langages permettent de créer des interfaces fluides et de gérer efficacement les appels asynchrones vers notre API via des bibliothèques comme Retrofit ou Volley.

JavaScript / React JavaScript est un langage de programmation incontournable pour le Web dynamique. Dans notre projet, il est utilisé côté client (navigateur) à travers la bibliothèque React. React permet de construire des interfaces utilisateur sous forme de composants réutilisables, ce qui est idéal pour développer notre tableau de bord administrateur (Single Page Application).

PHP PHP est un langage de script exécuté côté serveur, largement utilisé pour le développement web. Dans notre architecture orientée services, PHP est utilisé pour construire l'API RESTful. Il gère la logique métier, l'authentification (via les tokens JWT) et la communication avec la base de données de manière sécurisée.



Figure 5.1 : Langages de programmation utilisés (Java, JS, PHP, SQL).

5.1.2 Frameworks et Bibliothèques

React.js pour l'interface Web d'administration, React.js a été choisi pour sa rapidité et sa gestion efficace du DOM virtuel.

Volley (Android) Pour connecter nos applications mobiles à l'API distante, nous avons intégré des bibliothèques de requêtes HTTP (Retrofit ou Volley). Elles permettent de sérialiser et désérialiser facilement les données JSON échangées avec le serveur PHP.

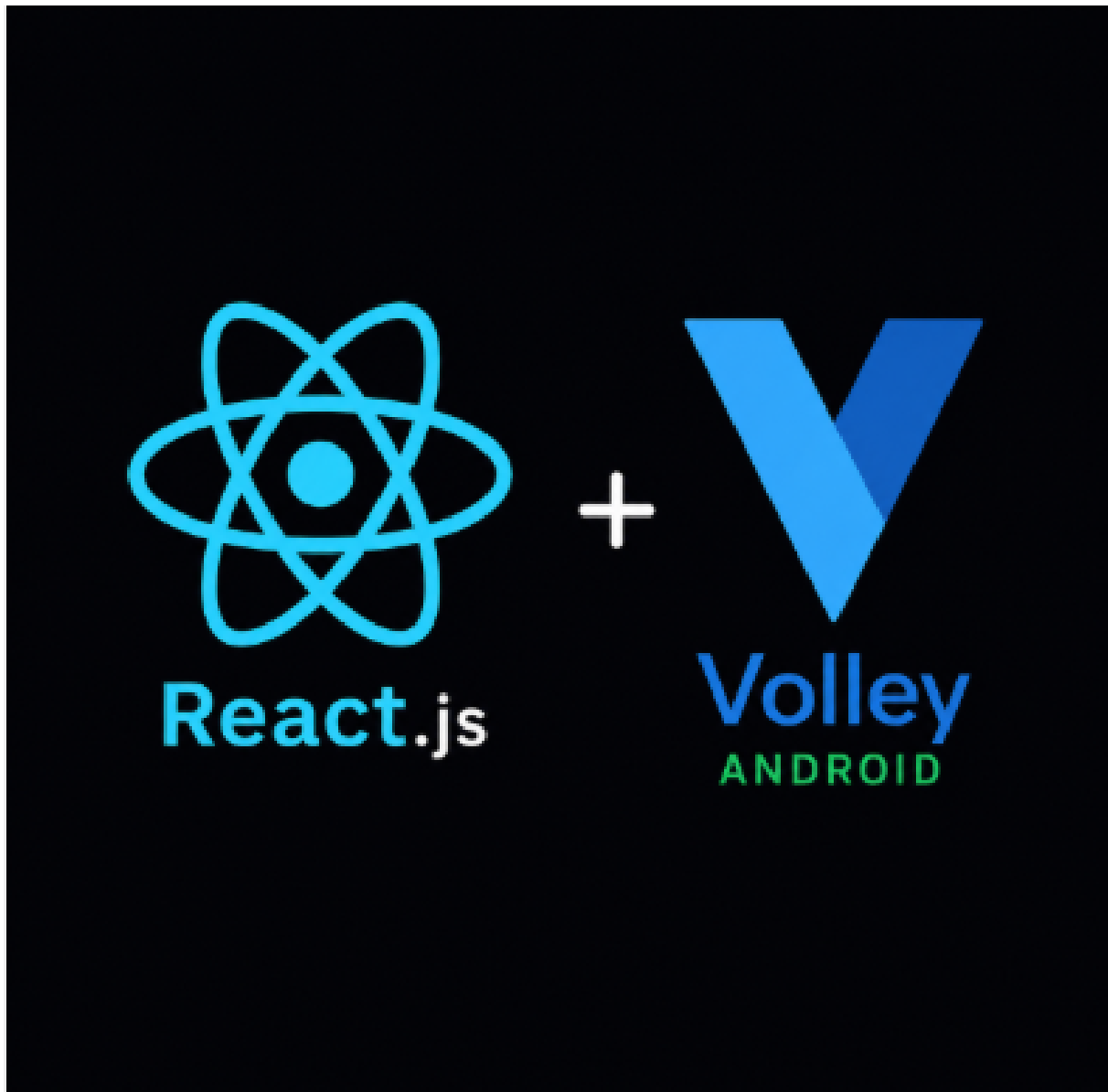


Figure 5.2 : Frameworks et bibliothèques (React, Volley).

5.1.3 Base de données

MySQL MySQL est un système de gestion de base de données relationnelle (SGBDR) open source. Notre projet respectant une contrainte stricte de 4 tables (Users, Addresses, Deliveries, Delivery_status_logs), MySQL offre la robustesse nécessaire pour gérer les clés étrangères et garantir l'intégrité de l'historique des statuts de livraison.



Figure 5.3 : Système de gestion de base de données relationnelle MySQL.

5.1.4 Critères de sélection

Les choix technologiques du projet ont été guidés par les exigences du cahier des charges :

- **Approche multi-plateforme** : Une API centrale (PHP) servant à la fois une application Mobile (Android) et une interface Web (React).
- **Sécurité** : L'utilisation de tokens JWT pour sécuriser les endpoints de l'API.
- **Simplicité et contraintes** : Une base de données relationnelle légère (MySQL) limitée à 4 tables pour se concentrer sur la logique des statuts de livraison.

5.2 Environnement de développement logiciel

Android Studio :

L'environnement de développement intégré (IDE) officiel de Google pour le développement d'applications Android. Il fournit les outils nécessaires pour concevoir les maquettes XML, écrire le code métier, et émuler des appareils mobiles pour les tests.

Visual Studio Code :

VS Code est un éditeur de code léger et puissant utilisé pour développer à la fois notre API Backend en PHP et notre interface d'administration Web en React.

XAMPP :

XAMPP est une suite de logiciels permettant de mettre en place facilement un serveur web local. Il intègre Apache et MariaDB/MySQL, essentiels pour héberger et tester notre API PHP et notre base de données durant la phase de développement.

Postman :

Postman est un outil indispensable pour le développement d'API. Il nous a permis de tester, documenter et valider tous nos endpoints REST (GET, POST, PUT) avant même que les interfaces front-end (Mobile et Web) ne soient terminées.

5.3 Identité visuelle du projet

Voici le logo de notre application :



Figure 5.4 : Logo pour notre application de livraison.

Le nom "*LocalTrack Delivery*" reflète la mission principale de notre plateforme : offrir une solution de suivi logistique de proximité. Conscients que les petites structures et les campus manquaient d'outils simples pour gérer leurs coursiers, nous avons conçu une identité visuelle épurée. Les couleurs (souvent dominées par des teintes de bleu ou d'orange) symbolisent la rapidité, la confiance et la clarté de l'information (statuts en temps réel).

5.4 Implémentation et interfaces

Authentification globale :

L'accès à la plateforme est sécurisé par un système d'authentification centralisé par JWT (JSON Web Token). L'API vérifie le rôle de l'utilisateur lors de la connexion pour le diriger vers l'interface appropriée : Mobile pour les Clients et Livreurs, Web pour l'Administrateur.



Espace Client (Mobile Android)

Après connexion, le client accède à son espace où il peut effectuer une nouvelle demande de livraison. L'interface lui permet de sélectionner une adresse de départ et une adresse de destination.

Une fois la livraison créée, le client accède à l'écran de suivi. C'est le cœur de l'expérience client : il peut voir en temps réel l'évolution de son colis ("Créé", "Prise en charge", "En route", "Livrée") grâce aux requêtes effectuées vers notre API.

20:29

63





Nouvelle Livraison

Campus Delivery



DÉTAILS DE LA LIVRAISON

 Adresse de départ



 Adresse d'arrivée

 Date (AAAA-MM-JJ)

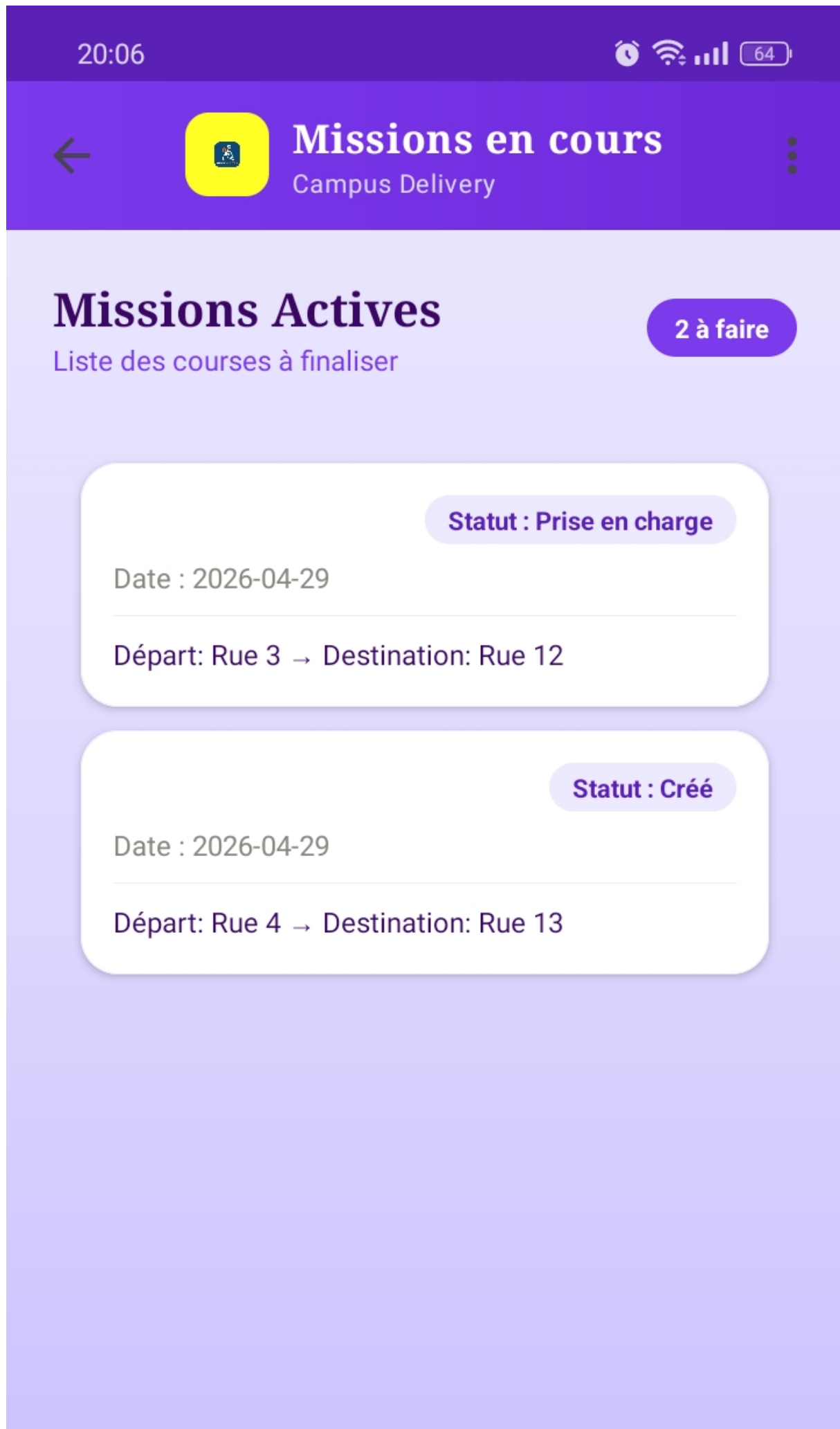
Confirmer la livraison

Votre livraison sera confirmée sous peu

Espace Livreur (Mobile Android)

Le livreur dispose d'une interface optimisée pour la mobilité. Son écran principal affiche la liste des livraisons qui lui sont affectées. En cliquant sur un élément, il accède aux détails (adresses).

L'action principale du livreur consiste à appuyer sur des boutons d'état (ex : "Marquer comme En Route" ou "Confirmer la Livraison"). Ces actions déclenchent des requêtes API (méthode PUT) qui mettent à jour la table `livraisons` et ajoutent une ligne dans `delivery_status_logs`.



Espace Administrateur (Web React)

L'administrateur, depuis son navigateur, accède à un tableau de bord (Dashboard) développé en React. Ce tableau agrège les données de l'API pour offrir une vision globale.

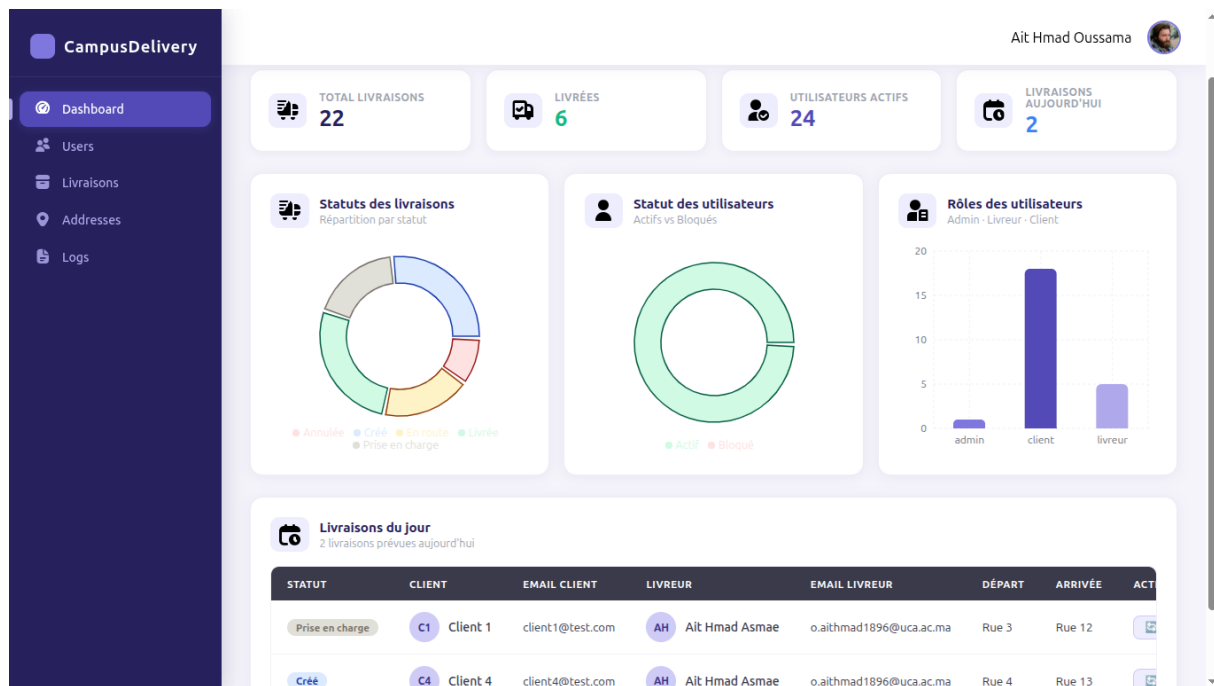


Figure 5.8 : Tableau de bord Admin Web (Statistiques et liste globale).

Ce tableau de bord présente des indicateurs clés, tels que le **nombre de livraisons du jour**, les **colis en attente**, et l'**activité des livreurs**. L'interface permet également des opérations CRUD complètes : l'administrateur peut visualiser toutes les livraisons, annuler une commande problématique, ou affecter manuellement un livreur à une course en attente.

5.5 Stratégie de tests

Pour garantir la fiabilité de notre système de suivi, une stratégie de tests a été mise en place en se concentrant sur les flux de données entre le Mobile, l'API et la Base de données.

5.5.1 Tests de l'API (Postman)

Avant de développer les interfaces, nous avons constitué une collection Postman complète.

- **Sécurité** : Validation de la génération des tokens JWT et de l'interdiction d'accès aux routes protégées sans token.
- **Validation métier** : Vérification que l'API refuse la création d'une livraison s'il manque l'adresse de départ ou d'arrivée.
- **Historisation** : Test unitaire vérifiant que chaque appel à la route de modification de statut ajoute bien une ligne dans la table `delivery_status_logs`.

5.5.2 Tests d'intégration Mobile

- **Gestion des états réseau** : Vérification du comportement de l'application Android en cas de perte de connexion (affichage d'un message d'erreur clair à l'utilisateur, chargement via ProgressBar).

- **Navigation et UX** : Validation des flux entre la Bottom Navigation et l’affichage des détails (Retrofit/Volley callbacks).

5.5.3 Tests de bout-en-bout (Scénario global)

Nous avons testé le cycle de vie complet d’une livraison :

1. Le Client A crée une livraison (Mobile).
2. L’Admin B observe la nouvelle livraison sur le Web (React) et vérifie son apparition.
3. Le Livreur C se connecte (Mobile), voit la livraison, et change le statut en "En route".
4. Le Client A actualise son application et constate le changement de statut en temps réel.

Ce test a permis de valider la parfaite communication entre la base de données (limitée à 4 tables) et les différentes interfaces.

5.6 Conclusion

Au cours de ce chapitre, nous avons abordé tous les aspects liés à la réalisation de notre plateforme de suivi de livraison locale. Nous avons présenté les technologies (Android, React, PHP, MySQL), ainsi que l’implémentation concrète des interfaces pour les trois acteurs du système.

L’architecture orientée services (API REST) a permis de découpler efficacement la logique serveur des interfaces clientes, tout en respectant la contrainte pédagogique des 4 tables en base de données. Les tests menés via Postman et les scénarios bout-en-bout garantissent la fiabilité de l’historisation des statuts de livraison. Cette réalisation technique représente l’aboutissement concret de l’analyse et de la conception des chapitres précédents, transformant notre cahier des charges en un produit fonctionnel et robuste.

Conclusion générale

Ce projet de fin du module porte sur la conception et la réalisation d'une plateforme de suivi de livraison locale, articulée autour d'une application mobile Android pour les acteurs de terrain (clients et livreurs) et d'une interface web React pour la supervision administrative. Nous avons suivi une démarche méthodologique rigoureuse, allant de la définition du cahier des charges et de la collecte des besoins fonctionnels jusqu'à la phase de conception et de mise en œuvre technique. Les principaux résultats de ce projet incluent une définition claire des exigences logistiques, une modélisation précise des interactions à travers des diagrammes de classes optimisés (4 tables maximum) et une architecture orientée services sécurisée.

Par ailleurs, l'intégration d'un système de suivi en temps réel permettant de tracer l'historique complet des statuts de livraison, ainsi que le développement d'une API REST fournissant une synchronisation fluide entre les différentes plateformes, ont été menés à bien. Cette réalisation ouvre encore des avenues d'amélioration et d'extension. En particulier, l'utilisation de nouvelles techniques d'optimisation de tournées (algorithmes de plus court chemin) et l'intégration de la géolocalisation GPS en temps réel pourraient encore renforcer les performances opérationnelles et la précision des délais de livraison annoncés. L'optimisation de la base de données et des algorithmes de traitement afin d'héberger un volume plus important de colis sans impacter la rapidité et l'efficacité du système constitue également un axe futur majeur.

Des techniques de personnalisation avancées seraient utilisables pour ajuster les notifications et adapter les recommandations d'affectation des livreurs en temps réel selon la charge de travail et la proximité géographique. De plus, le renforcement des mesures de sécurité, notamment pour sécuriser les données sensibles des clients (adresses, coordonnées) et assurer la conformité avec les réglementations actuelles sur la protection des données personnelles (RGPD), reste fondamental. Le développement d'options interactives comme l'intégration de signatures numériques à la livraison ou la coordination directe via une messagerie interne entre client et livreur procurerait une expérience utilisateur plus enrichissante et sécurisante.

Il reste cependant une question : comment pouvons-nous assurer que ces systèmes de suivi logistique non seulement répondent aux besoins immédiats de rapidité et d'efficacité, mais soutiennent aussi une gestion durable et éthique des livraisons sans ignorer, d'une part, des considérations environnementales (optimisation carbone) et, d'autre part, la transparence des algorithmes d'affectation des tâches aux travailleurs ? Cette question appelle une attention et une réflexion permanente sur l'équilibre entre automatisation logistique, autonomie des travailleurs et respect de la vie privée dans la réalisation et l'utilisation des outils de suivi connectés.

Apport du projet

Au début du projet, les objectifs étaient clairement définis vers la création d'une solution de gestion logistique capable de suivre en temps réel les livraisons locales et de coordonner les flux entre clients, livreurs et administrateurs. La démarche de recherche et d'expérimentation a permis de développer une approche structurée de résolution de problèmes dans le domaine des applications multi-plateformes orientées services.

Parmi les compétences développées, la maîtrise de l'écosystème mobile avec Android Java pour les interfaces de terrain, couplée à une interface d'administration en React et un backend API en PHP/MySQL, a permis de construire une architecture robuste et cohérente. L'implémentation de la logique de changement de statut et l'historisation systématique dans la table des logs ont représenté un défi technique majeur, garantissant la traçabilité des colis. L'adoption de la méthodologie Scrum a favorisé une organisation efficace par sprints, tandis que LaTeX a permis une documentation technique rigoureuse.

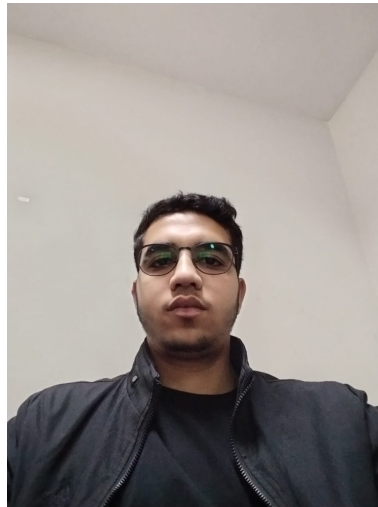
Ce projet a présenté des défis spécifiques, notamment dans la synchronisation en temps réel des données entre l'application mobile et le dashboard web, ainsi que dans la sécurisation des échanges via des tokens JWT. L'équilibrage entre la simplicité de l'interface mobile pour les livreurs et la complétude des informations pour l'administrateur a nécessité de nombreux ajustements ergonomiques.

Si le projet était à refaire, l'intégration de services de cartographie comme Google Maps API permettrait d'optimiser les trajets des livreurs, une architecture basée sur les WebSockets faciliterait les mises à jour instantanées sans rafraîchissement, et une phase de tests utilisateurs avec de vrais coursiers permettrait d'affiner encore davantage l'expérience utilisateur (UX) mobile.

Bibliographie

- [1] Le cycle de projet Scrum expliqué.
Nutcache Blog.
<https://www.nutcache.com/fr/blog/cycle-projet-Scrum/>.
- [2] Principes et architecture des API RESTful.
Red Hat Guide.
<https://www.redhat.com/fr/topics/api/what-is-a-rest-api>.
- [3] Introduction to JSON Web Tokens.
JWT.io.
<https://jwt.io/introduction>.
- [4] Documentation officielle Android pour les développeurs.
Google Developers.
<https://developer.android.com/>.
- [5] React : Bibliothèque JavaScript pour créer des interfaces utilisateur.
React Documentation.
<https://fr.reactjs.org/>.
- [6] Optimisation de base de données pour la logistique.
MySQL Documentation.
<https://dev.mysql.com/doc/>.
- [7] UML : un langage de modélisation de type graphique.
IONOS Digital Guide.
<https://www.ionos.fr/digitalguide/sites-internet/developpement-web/uml-un-langage-de-modelisation-pour-la-programmation-orientee-objet/>.
- [8] Apprendre le HTML : guides et didacticiels.
MDN Web Docs.
https://developer.mozilla.org/fr/docs/Learn/HTML/Introduction_to_HTML.
- [9] Qu'est-ce que le CSS ? Définition et application.
IONOS Digital Guide.
<https://www.ionos.fr/digitalguide/sites-internet/web-design/quest-ce-que-le-css/>.
- [10] JavaScript : qu'est-ce que c'est ?
Futura Sciences.
<https://www.futura-sciences.com/tech/definitions/internet-javascript-509/>.
- [11] PHP : Hypertext Preprocessor - Manuel Officiel.
<https://www.php.net/manual/fr/>.

- [12] MySQL, tout comprendre au logiciel de gestion de données relationnelles.
DataScientest.
<https://datascientest.com/mysql-tout-comprendre>.
- [13] Visual Studio Code - Le blog technique Webnet.
<https://blog.webnet.fr/visual-studio-code/>.
- [14] Apache Friends - About the XAMPP project.
<https://www.apachefriends.org/fr/about.html>.
- [15] Postman : The Collaboration Platform for API Development.
<https://www.postman.com/>.
- [16] StarUML - Logiciel de modélisation UML.
<https://fr.wikipedia.org/wiki/StarUML>.



Ait Hmad Oussama

Résumé du Projet

Le présent projet vise à développer une solution innovante destinée à optimiser la gestion des flux logistiques de proximité grâce à une plateforme de suivi de livraison intégrée. Cette solution permet une coordination fluide entre les clients, les livreurs et les administrateurs en offrant une visibilité en temps réel sur l'état des colis. Pour atteindre cet objectif, une approche méthodologique rigoureuse a été adoptée, combinant une analyse fonctionnelle approfondie, une modélisation précise du système (limitée à 4 tables pour garantir la performance) et l'implémentation de technologies modernes telles qu'Android (Java/Kotlin), React pour la supervision web, et une API REST en PHP.

Les contributions majeures du projet incluent la mise en œuvre d'interfaces mobiles intuitives pour les acteurs de terrain, l'intégration d'un système performant d'historisation des statuts de livraison (logs), ainsi que la validation de la solution à travers des tests de bout en bout garantissant l'intégrité des données. Ces résultats traduisent une amélioration significative en termes de réactivité opérationnelle, de transparence des informations et de fiabilité du suivi par rapport aux méthodes de gestion classiques.

Ce projet a permis d'acquérir des compétences avancées en développement mobile Android, en création d'interfaces web réactives (React), en sécurisation d'API (JWT) et en architecture logicielle orientée services. En outre, il ouvre des perspectives prometteuses pour de futurs développements, tels que l'optimisation des tournées par géolocalisation, l'intégration de notifications push pour les changements de statut, et l'enrichissement des tableaux de bord statistiques pour une aide à la décision plus poussée.

L'ensemble de ces réalisations témoigne de l'engagement à proposer une solution à la fois innovante, évolutive et efficace, répondant aux problématiques de logistique urbaine identifiées dès les premières phases du projet.

Mots clés : Suivi de livraison, Gestion des statuts, Logistique locale, interface utilisateur, gestion des données , Optimisation des flux.